

Введение

В программировании часто возникает необходимость создавать новые объекты, которые уже имеют заранее определённые характеристики. При этом создание объекта через конструктор может быть затратным или неудобным.

Для решения данной задачи используется шаблон проектирования **«Прототип» (Prototype)**. В рамках данного практического задания была реализована программа на языке Python, демонстрирующая создание объектов путем их клонирования.

Основная идея работы — создавать новые объекты на основе уже существующих, копируя их состояние, что делает процесс создания более быстрым и удобным.

Цель работы

Целью данной работы является:

- изучение паттерна «Prototype»;
- реализация механизма клонирования объектов;
- закрепление навыков объектно-ориентированного программирования;
- понимание различий между поверхностным и глубоким копированием;
-

Теоретическая часть

Паттерн **Prototype (Прототип)** — это порождающий шаблон проектирования, который позволяет создавать новые объекты путем копирования уже существующих экземпляров.

Суть паттерна заключается в том, что:

- создается объект-прототип;
- затем он копируется;
- новый объект получает те же свойства, что и оригинал.

Преимущества паттерна:

- уменьшение затрат на создание объектов;
- упрощение кода;
- отсутствие зависимости от конкретных классов;
- возможность создавать объекты во время выполнения программы.

Недостатки:

- сложность реализации глубокого копирования;
- возможные ошибки при работе с вложенными объектами.

Постановка задачи

Согласно заданию, необходимо реализовать систему биржи фриланса, в которой есть два типа пользователей:

- исполнитель (Executor);
- заказчик (Customer).

Каждый пользователь имеет:

- идентификатор;
- ФИО.

Дополнительно:

- у заказчика есть объект паспортных данных (Passport);
- необходимо реализовать метод `Clone()`;
- для исполнителя используется поверхностное копирование;
- для заказчика — глубокое копирование, так как есть вложенный объект.

Принцип работы программы

1. Создается объект исполнителя;
2. Выполняется его клонирование;
3. Создается объект заказчика с паспортом;
4. Выполняется клонирование заказчика;
5. Проверяется, что:
 - исполнитель копируется корректно;
 - заказчик получает новый независимый объект паспорта.

Заключение

В ходе выполнения практической работы был изучен и реализован паттерн «Прототип». Данный шаблон позволяет создавать новые объекты путем копирования существующих, что значительно упрощает процесс разработки.

Практическая реализация показала, что важно правильно выбирать между поверхностным и глубоким копированием, особенно при работе с вложенными объектами.

Полученные знания могут быть использованы при разработке систем, где требуется быстрое создание множества объектов с похожими характеристиками.

```
from abc import ABC, abstractmethod
import copy
```

```
# =====
# Класс паспортных данных
# =====
class Passport:
    def __init__(self, number: str, country: str):
        self.number = number
        self.country = country

    def __str__(self):
        return f"Паспорт: {self.number}, Страна: {self.country}"
```

```
# =====
# Абстрактный класс пользователя
# =====
class User(ABC):
    def __init__(self, user_id: int, full_name: str):
        self.user_id = user_id
        self.full_name = full_name

    @abstractmethod
    def clone(self):
        pass

    def __str__(self):
        return f"ID: {self.user_id}, ФИО: {self.full_name}"
```

```
# =====
# Исполнитель
# Поверхностное копирование
# =====
class Executor(User):
    def clone(self):
        return copy.copy(self)

    def __str__(self):
        return f"Данные исполнителя: {super().__str__()}"
```

```
# =====
# Заказчик
# Глубокое копирование паспорта
# =====
class Customer(User):
    def __init__(self, user_id: int, full_name: str, passport: Passport):
        super().__init__(user_id, full_name)
        self.passport = passport
```

```

def clone(self):
    new_passport = Passport(self.passport.number, self.passport.country)
    return Customer(self.user_id, self.full_name, new_passport)

def __str__(self):
    return f"Данные заказчика: {super().__str__()}, {self.passport}"

# =====
# Клиентский код
# =====
if __name__ == "__main__":
    # Создаем исполнителя
    executor1 = Executor(1, "Иванов Иван Иванович")
    executor2 = executor1.clone()

    print("Исполнитель (оригинал):")
    print(executor1)
    print("Исполнитель (клон):")
    print(executor2)
    print("-" * 50)

    # Создаем заказчика
    passport1 = Passport("N12345678", "Казахстан")
    customer1 = Customer(2, "Петров Петр Петрович", passport1)
    customer2 = customer1.clone()

    print("Заказчик (оригинал):")
    print(customer1)
    print("Заказчик (клон):")
    print(customer2)
    print("-" * 50)

    # Проверка, что паспорта разные объекты
    print("Проверка глубокого копирования:")
    print("customer1.passport is customer2.passport ->", customer1.passport is
customer2.passport)

```